

TCN

Motivation & Problem Setup

- 현재까지 거의 대부분의 딥러닝 사용자들에게 sequence modeling = rnn처럼 여겨져 왔다.
- 그러나, 최근 연구 결과는 cnn으로 audio, machine translation 같은 sequence modeling task에서 rnn을 앞설 수 있다고 말한다. sequence modeling의 특정 분야에 국한되어 cnn의 성공적인 성능이 나오는 것인지 혹은 sequence modeling에서 rnn의 독점을 다시 고려해 볼 만한 이슈인지 자세히 알아볼 필요가 있다.
- 이 논문에서는 이를 알아보기 위해 rnn이 우세하게 사용되고 있는 다양한 sequence modeling benchmark에서 cnn과의 체계적인 비교 실험을 진행한다.
- Sequence modeling에서 활용될 수 있는 cnn의 대표 모델 TCN을 제안했음. TCN은 불과 몇 년 전 나왔으며 의도적으로 심플함을 유지함과 동시에 여러 task에서 성공적이었던 cnn 구조를 결합했다.
- 논문에서의 결과는 TCN 아키텍처가 다양한 task에서 기존 lstm 같은 recurrent based model을 앞섰고, 또한 longer effective memory(장기 기억 능력)를 입증했다. rnn design에 초점을 맞춘 다양한 benchmark를 포함하고 있는 결과이기 때문에 이는 매우 주목할 만한 성과라고 볼 수 있다.
- 나아가 여러 가지 결과를 잘 해석하기 위해 recurrent network의 기억력 저하 문제에 대해서 더 분석했다. 기존에 rnn에서 이론적으로 입증된 장기 기억 능력에도 불구하고 TCN은 이를 넘는 longer effective memory를 보였다.
- 일반적으로 sequence modeling == rnn에 대한 통념은 다시 생각해 볼 필요가 있다. sequence modeling에서 발휘되는 cnn의 성능을 무시하지 말자.

Method

Overall Architecture of TCN

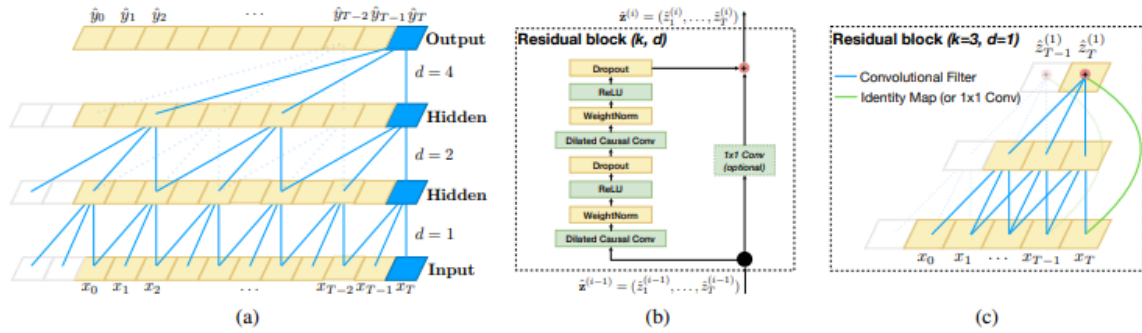


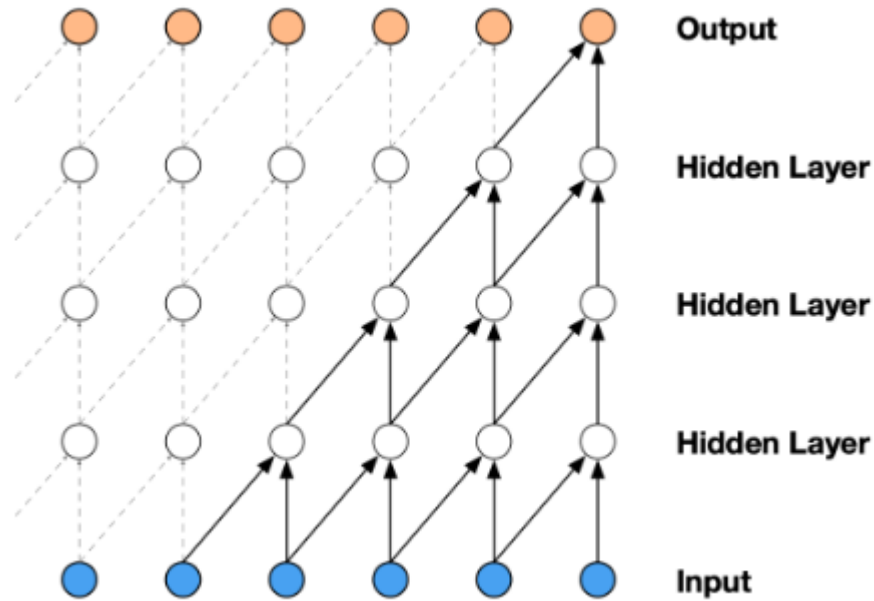
Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. An 1×1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

- TCN은 Causal convolution으로 구성된 아키텍처이다. 이는 미래에서부터 과거로의 정보 유출이 없다는 것을 뜻한다. TCN은 기본적으로 1d-dilated causal convolutional network를 사용하는 wavenet에 기반하지만 layer 간의 skip connection을 없애거나 gate activations 등을 없애면서 훨씬 가벼워진 모형이다.
- TCN은 input으로 다양한 길이의 sequence를 받고 이를 동일한 길이의 output sequence로 맵핑시킨다. (RNN과 같은 방식)

Sequence Modeling

- Input sequence: $x_0, \dots, x_T$ 를 받아 각 시간에 대응하는 output: $y_0, \dots, y_T$ 를 예측하고자 함. 이 때, y_t 를 예측할 때는 t 시점 포함 이전의 input ($x_0, \dots, x_t$)만을 활용한다.
- $f: X_{T+1} \rightarrow Y_{T+1}$ 의 함수로 sequence modeling network를 표현할 수 있겠다. 이는 y_t 가 오직 t 보다 과거 시점의 input $x_0, \dots, x_T$ 에만 의존한다는 causal constraint에 기반한다.
- 예측 값과 실제 값 사이의 오차 $L(y_0, \dots, y_T, f(x_0, \dots, x_T))$ 를 최소화하는 function f 를 찾는 것이 최종 목표.

Causal Convolution



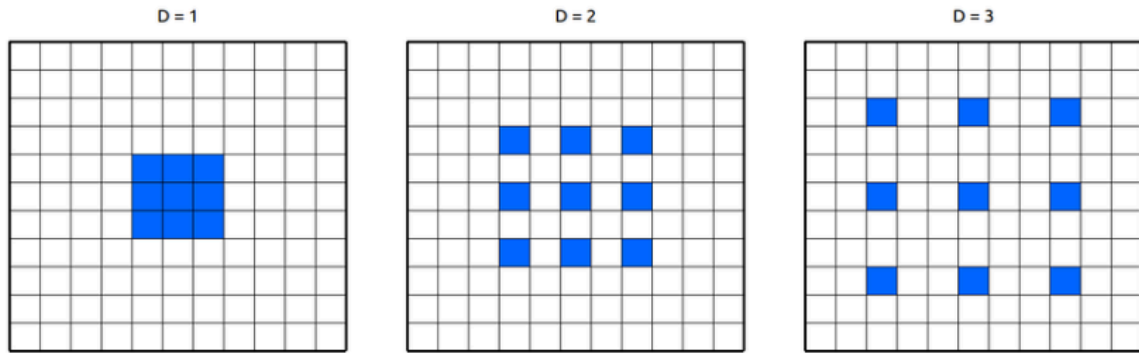
TCN은 두 가지의 원리에 기반하는데, 우선 network가 input과 동일한 길이의 output을 내놓는다는 것이고 다른 하나는 미래에서 과거로의 정보 유출이 없다는 것이다(causal). 즉 convolution 연산시에 time step t 의 output을 내기 위해서는 time t 와 그 이전 시점의 원소들과만 convolved 된다는 것이다.

- 첫 번째 특성을 만족시키기 위해서 TCN은 output hidden layer가 input layer와 동일한 길이를 갖는 1D fully-convolutional network(FCN)을 사용한다. 이전 layer와 동일한 길이를 유지하기 위해서는 (kernel size - 1) 만큼의 zero padding이 추가된다. dilation이 추가된다면 zero padding은 $(\text{kernel_size}-1) \times \text{dilation factor}$
- 두 번째의 causal한 특성을 만족시키기 위해서 TCN은 t 시점에서의 convolution 연산이 이전 layer의 t 시점과 그 이전의 시점들로만 구성되어 있는 Causal convolution 연산을 사용한다.

TCN = 1D FCN + Causal Convolutions

FCN + Causal Convolutions로만 이루어진 basic design은 긴 길이를 갖는 sequence에서 long effective history를 얻기 위해 매우 깊은 network를 사용하거나 큰 filter size를 사용해야 한다는 단점이 존재한다. 따라서 Dilated convolutions를 사용하게 됨.

Dilated Convolutions



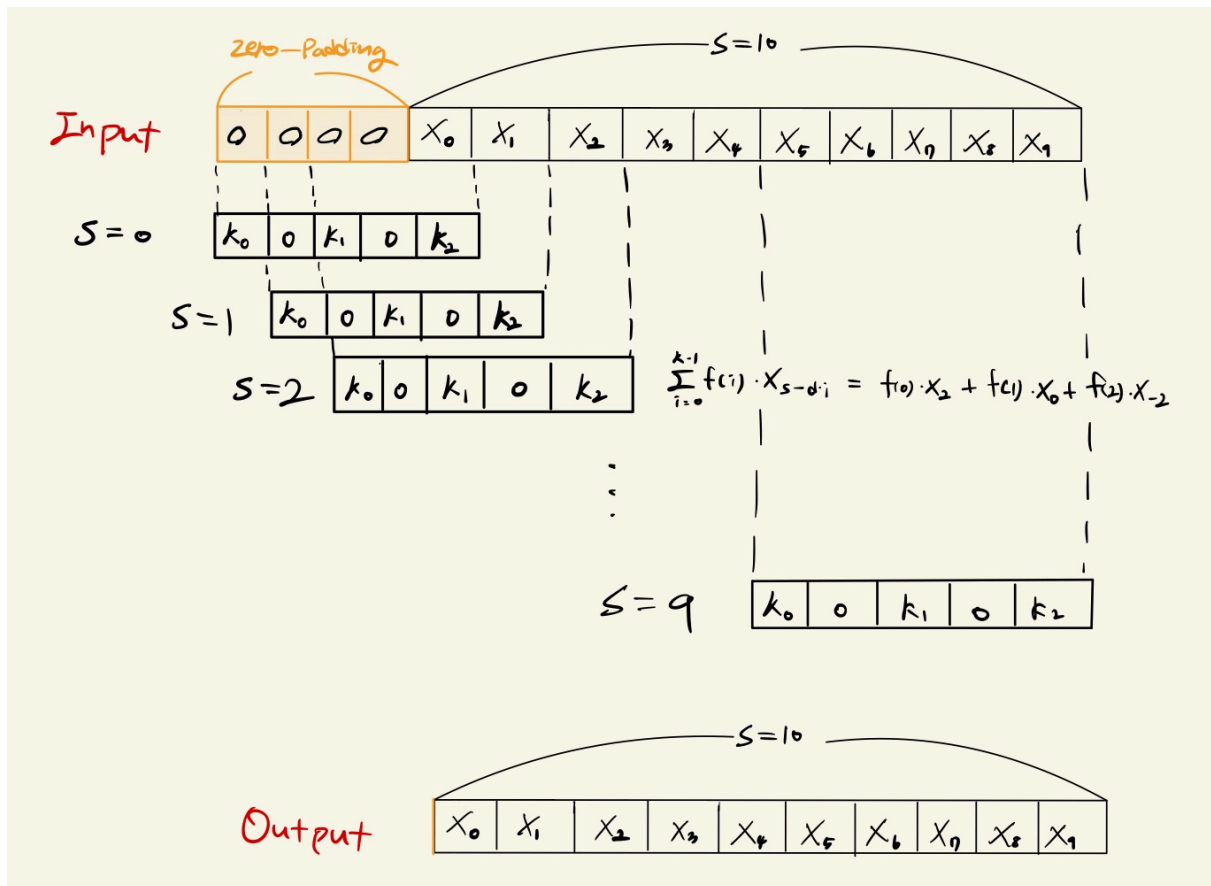
- Receptive Field란 filter가 한 번에 보는 영역. Dilated convolutions는 Receptive Field를 크게 만들면서 연산량의 증가를 가져오지 않는 효과적인 방법.
- Simple한 causal convolution에서는 layer의 깊게 쌓아도 큰 receptive field를 만드는 것에 한계가 있다. 이는 먼 과거에 영향을 받는 sequence 데이터를 다루는 것을 어렵게 만든다. 반면, dilated convolution에서는 layer가 깊어짐에 따라서 receptive field가 지수적으로 증가할 수 있게 만들어준다.

- 1-D sequence input $x \in \mathbb{R}^n$, filter $f: \{0, \dots, k-1\}$ 에서의 dilated convolution operation

$$F(s) = (x * df)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i}$$

s : sequence, d : dilation factor, k : filter size

1D Dilated Causal Convolutions example



kernel size=3, sequence length=10, dilation factor=2에서의 TCN 연산 예시.
(Dilated causal convolution)

- TCN의 receptive field를 늘리는 방법이 두 가지 있다. 우선 첫 번째로 filter size k 를 늘리고, dilation factor d 를 증가시킴으로써 receptive field를 늘릴 수 있다. 예를 들어 충분히 큰 filter size k 와 dilation factor d 를 설정하였다면, 하나의 layer만으로도 $(k-1) \cdot d$ 의 receptive field를 확보하게 되는 것. 두 번째로는 network의 깊이를 깊게 쌓는 것인데 보통 dilation factor d 를 layer의 깊이에 맞춰 지수적(2^i , i : depth of network)으로 증가하는 형식을 많이 사용한다.
- receptive field를 늘리는 것은 아주 오래전의 과거에 영향을 받는 sequence data의 특성을 잘 반영하게 해주고 예측 성능을 높일 수 있는 방법이다. - **large effective history**

Residual Connections

- $o = \text{Activation}(x + F(x))$
- Residual connection은 이미 깊은 network에서의 degradation 문제를 해결하며 좋은 성능을 보여왔음.

- TCN에서는 특히 receptive field의 확보를 위해 network의 깊이, filter size, dilation factor에 의존하고 있으므로 모델 사이즈가 커지는 것은 불가피하다. 따라서 깊은 layer의 network에서 학습 안정화는 중요한 포인트이다.
- TCN Residual Block에서는 input과 output이 다른 channel 개수를 갖고 있을 수 있다. 따라서 추가적으로 1*1 convolution을 적용함으로써 output과 더해지는 input의 channel size를 맞출 수 있다.

Discussion

Advantages

- **Parallelism:** RNN 계열의 모델들과 다르게 병렬화 가능
- **Flexible receptive field size:** TCN에서는 dilated conv layer를 깊게 쌓거나, 큰 dilation factor를 사용하거나, filter size의 크기를 늘리는 방식으로 receptive field의 사이즈를 조절하는 것이 가능하다. 이는 자유롭게 모델의 메모리 사이즈 조절 및 도메인 별 설정을 유연하게 가져갈 수 있다.
- **Stable gradients:** TCN에서는 sequence의 temporal한 방향과는 다르게 backpropagation이 이루어진다. 따라서 rnn계열의 모델에서 흔히 발생하는 gradient vanishing이나 gradient exploding현상을 피할 수 있다.
- **Low memory requirement for training:** Sequence의 길이가 길 때, lstm이나 gru모델에서는 다수의 cell gate 속에 많은 정보들을 저장하게 된다. 하지만 TCN의 경우 같은 layer에서 weight filter들끼리 가중치를 공유하기 때문에 메모리 사용을 절약할 수 있다.
- **Variable length inputs:** RNN과 동일하게 모델이 다양한 길이의 input을 받을 수 있다. 이는 TCN이 sequence를 sliding하며 1D convolution을 적용하기 때문이다.

Disadvantages

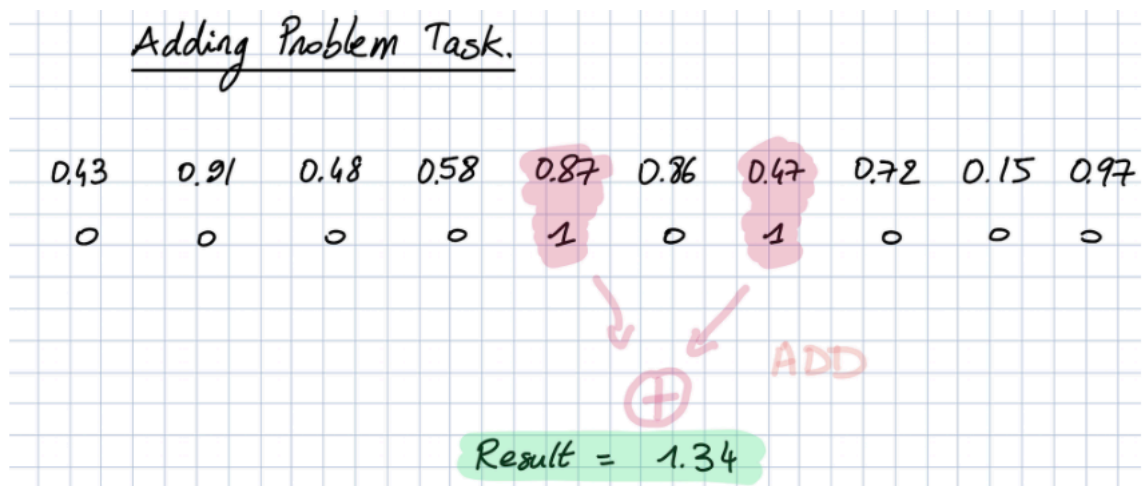
- **Data storage during evaluation:** RNN계열의 모델들은 $t+1$ 시점의 예측을 수행할 때, t 시점의 input과 hidden, cell state만 유지하면 된다. t 시점 이전의 sequence input들은 더이상 유의미하지 않기 때문에 처분함으로써 메모리 효율을 높일 수 있다. 하지만, TCN은 예측 수행시에도 전체 raw sequence가 연산에 활용되므로 메모리 사용이 요구된다.
- **Potential parameter change for a transfer of domain:** TCN의 receptive field의 크기는 k 와 d 의 값으로 결정된다. 만약 작은 k 와 d 값의 TCN으로 학습된 모델이 다른 도메인에 전이되어 학습(transfer learning)된다고 할 때, 충분한 receptive field

를 반영하지 못해 성능이 좋지 않을 수도 있다. 즉, 얼마나 먼 시점의 과거로부터 영향을 받는 sequence 데이터인지에 따라서 모델의 가용 범위가 결정된다.

Experiment

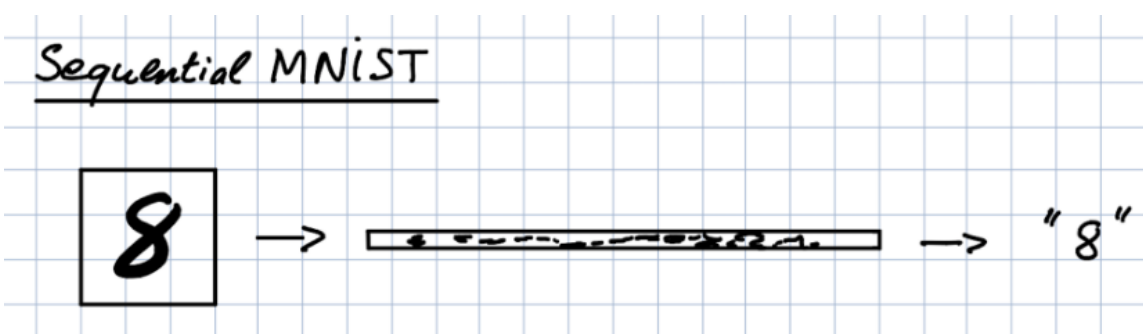
Dataset

- The adding problem



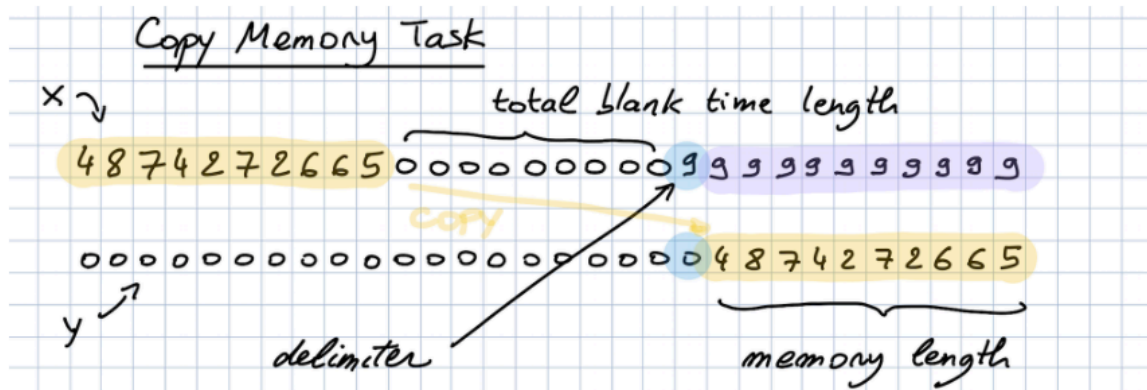
길이 n의 sequence 데이터인데 2개의 차원을 가짐. 첫 번째 dimension 벡터는 0~1 사이의 값을 갖는 random value. 두 번째 dimension의 벡터에서는 두 개의 원소를 제외하고는 전부 0. 목적은 두 번째 차원의 벡터에서 1로 지정된 위치의 random value의 합을 구하는 것으로 설정.

- Sequential MNIST and P-MNIST



Sequential MNIST는 RNN이 먼 과거의 정보를 기억하는 정도를 측정하기 위해서 많이 사용된 benchmark이다. MNIST는 flatten되어 784길이의 sequence가 되고 0부터 9까지의 숫자를 맞추는 classification문제가 된다. P-MNIST는 sequence가 랜덤하게 섞인 것으로 더 어려운 과제임.

- Copy memory



Input sequence는 $T+20$ 의 길이를 갖는다. 첫 번째 10개의 값은 1~8사이의 값으로 랜덤하게 지정되고 마지막 11개의 값은 9로 채워지는데 첫 번째 9의 위치는 delimiter 역할을 하게됨. 그리고 나머지 모든 원소들은 0으로 채워진다. 우선 이 benchmark 실험에서는 input과 같은 길이의 output을 생성한다. 이 때, 모델은 delimiter 위치까지의 모든 원소를 0으로 만들고 delimiter 이후 10개의 값으로 input의 처음 10개 원소 (1~8 사이의 랜덤 값)를 맞추어야 한다.

- JSB Chorales and Nottingham: polyphonic music dataset
- PennTreebank: text dataset
- Wikitext-103: text dataset
- LAMBADA: text dataset
- text8: text dataset

Experimental Setup

- RNN 모델의 홈구장같은 곳에서(주로 RNN이 실험되는 benchmark) TCN과 RNN의 성능을 비교 실험. 여러 도메인의 real data와 synthetic data를 포괄적으로 활용.
- TCN에서 network depth n , kernel size k 를 달리하여 여러가지 버전으로 실험. exponential dilation $d=2^i$ (i : network depth)
- Grid search for training RNN, network 사이즈는 TCN과 거의 동일하게 설정.
- Gating mechanism, skip connection과 같은 정교한 아키텍처 작업은 RNN, TCN 모두 추가되지 않음. (vanilla model)

Result

- The adding problem

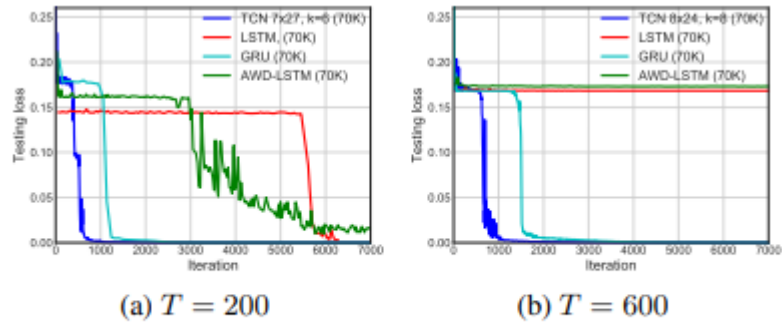


Figure 2. Results on the adding problem for different sequence lengths T . TCNs outperform recurrent architectures.

problem size 200, 600에서 실험되었고 모든 모델은 대략 70K의 파라미터 사이즈를 갖는다. 결과를 보면 TCN의 수렴이 가장 빠르고 GRU도 TCN보다 수렴속도는 느리지만 괜찮은 성능이다. 반면, LSTM과 RNN은 problem size 600세팅에서 loss가 수렴하지 않는 모습.

- Sequential MNIST and P-MNIST

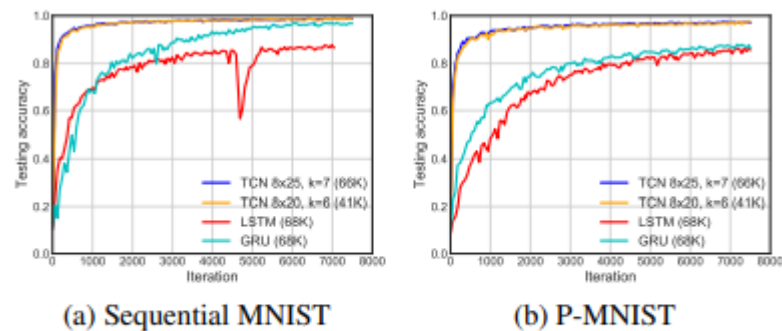


Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

TCN이 수렴 속도와 최종 accuracy에서 RNN 계열의 모델들을 압도적으로 앞섬.

- Copy memory

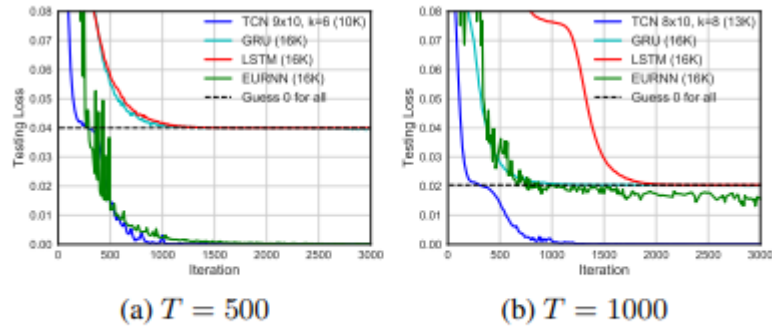


Figure 4. Result on the copy memory task for different sequence lengths T . TCNs outperform recurrent architectures.

해당 task에서 두각을 나타낸 RNN 계열의 모델 EURNN을 추가적으로 넣어서 실험. $T=500$ 실험에서는 두 모델이 거의 비슷한 수렴 속도를 보였지만, $T=1000$ 실험에서 TCN이 조금 앞선다. (loss와 수렴비율 모두)

- Memory Size of TCN, RNN

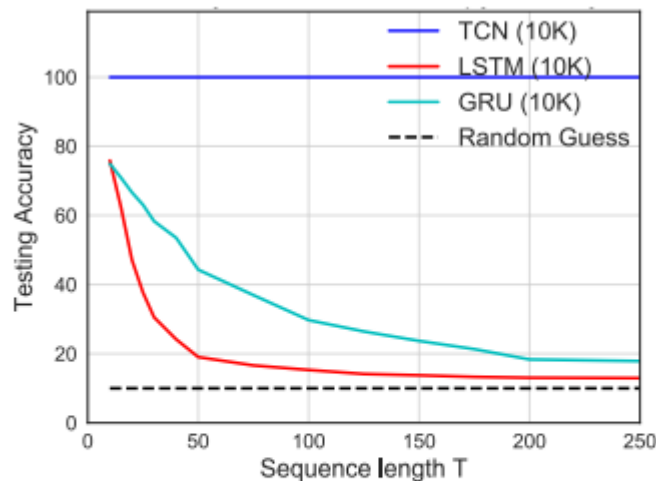


Figure 5. Accuracy on the copy memory task for sequences of different lengths T . While TCN exhibits 100% accuracy for all sequence lengths, the LSTM and GRU degenerate to random guessing as T grows.

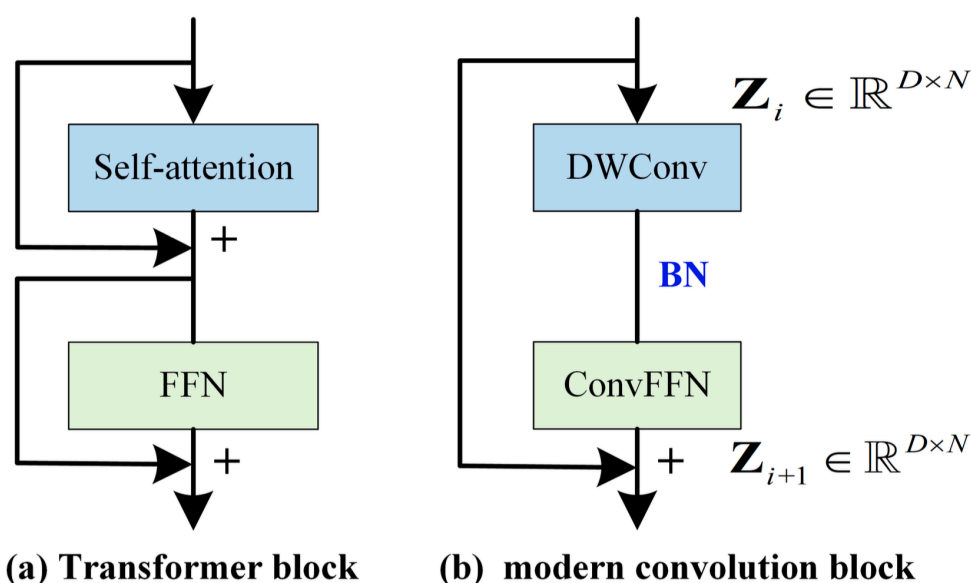
무한한 길이의 sequence에서 RNN과 TCN의 기억 보유 능력을 실험하기 위해서 다양한 T (sequence length)에서의 copy memory task 정확도를 측정했다. TCN과 RNN 모두 parameter size 10K에서 실험했다. 위의 결과를 보면 T 의 길이가 250이 될 때까지 TCN은 test 정확도 100%에 수렴한다. 이는 RNN 비교실험 모델보다 TCN이 더 긴 유효한 기억 능력을 갖고 있다는 것을 입증한다.

Introduction

시계열 예측은 다양한 적용 범위를 가지는 기술입니다. 결측치 예측, 행동 예측, 이상치 탐지 등 많은 활용이 가능합니다. 최근 시계열 분석 및 예측 분야에서 트랜스포머와 MLP 기반 모델들이 두각을 나타내면서 컨볼루션 기반 모델들이 소외되었습니다. 그러나 컨볼루션 기반 모델은 효율과 성능의 밸런스가 좋기 때문에 이 저자는 완전 컨볼루션 모델을 연구하고자 했습니다.

트랜스포머와 MLP 기반 모델의 성능이 좋았던 것은 **ERFs(effective receptive fields)**가 전역적이기 때문에 장기적인 시간 의존성(시간적으로 멀리 떨어진 두 시점 사이의 관계)을 더 잘 포착할 수 있기 때문입니다. TCN(Temporal Convolution Network)은 시계열 예측 분야에서 사용하는 컨볼루션 딥러닝 모델입니다. 기존에는 TCN이 트랜스포머 및 MLP 기반 모델의 성능을 따라잡기 위해 컨볼루션 부분을 업그레이드하는 것보다 복잡한 구조를 덧붙이는 것에 집중했습니다. 이는 ERF를 향상시키는 것을 통해 TCN의 성능을 올릴 수 있다는 판단이 있었기 때문입니다.

반면 CV 분야에서는 컨볼루션 자체를 최적화하는 모던 컨볼루션(modern convolution)을 고안해냈습니다. 모던 컨볼루션은 트랜스포머의 아키텍처 디자인을 차용하여 만들어졌습니다. 게다가 컨볼루션은 변수간 의존성을 잘 포착할 수 있는 구조입니다. 따라서 저자는 이 구조를 시계열 예측에서도 사용해보고자 했습니다.



(좌) 트랜스포머 블록의 구조, (우) 모던 컨볼루션 블록의 구조

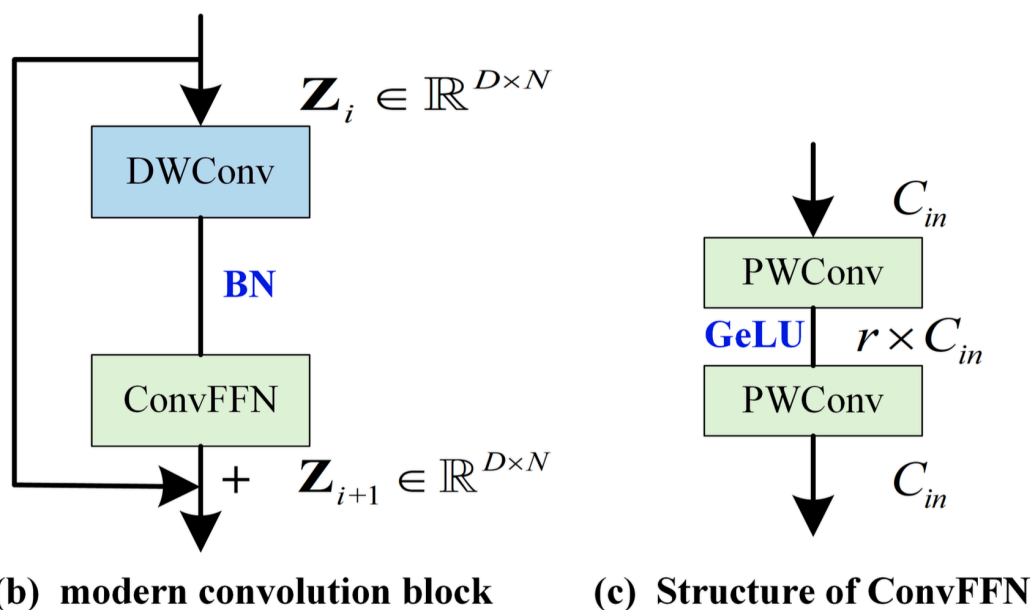
Related Work

시계열에서 다시 컨볼루션을 사용하기 위한 노력이 없었던 것은 아닙니다. 예를 들어, MICN은 여러 스케일의 컨볼루션 구조를 사용해 지역적인 특징과 전역적인 특징을 갖는 아키텍처

이고, SCINet은 인과적 컨볼루션(현재 시점의 출력을 계산할 때 미래의 정보를 사용하지 않도록 보장하는 컨볼루션 방식) 개념을 버리고 재귀적 다운샘플링-컨볼루션-상호작용 아키텍처를 사용했습니다. 그러나 ERF의 한계는 여전히 존재했고 장기 의존성을 모델링하기는 어려웠습니다. 반면 TimesNet은 1D 컨볼루션을 사용하는 공식을 깨고 시계열을 2D 행렬로 변환하여 CV처럼 2D 컨볼루션을 사용했으며 여러 시계열 예측 분야의 SOTA 모델로 자리 잡고 있습니다.

ModernTCN

Modern Convolution



(좌) 모던 컨볼루션 블록의 구조 (우) ConvFFN의 디테일한 구조

모던 컨볼루션 블록은 크게 세 가지 구조가 있습니다: DWConv 모듈, ConvFFN 모듈, 그리고 스킵 커넥션. DWConv(Depthwise Convolution)는 feature 단위로 시간적인 정보를 학습합니다. 즉, 트랜스포머의 셀프어텐션 모듈과 같은 역할을 합니다. ConvFFN은 트랜스포머의 FFN 모듈과 비슷한 역할을 합니다. ConvFFN은 두 개의 PWConv(Pointwise Convolution)와 inverted bottleneck 구조로 구성됩니다. PWConv는 채널간의 상호작용을 학습하는 구조이고 inverted bottleneck 구조는 모던 컨볼루션에 데이터가 입력되기 전 변수의 차원(수)를 줄였던 것을 다시 원래의 차원으로 확장하기 위한 역할을 합니다. 위 디자인을 이용하면 시간적 정보와 feature mixing을 분리하여 학습할 수 있습니다.

그러나 이런 구조가 시계열 예측에서 성능 향상이 미미했다고 합니다. 다변량 시계열 데이터에는 피쳐 차원, 시간 차원에 더해 변수 차원이 있습니다. 모던 컨볼루션 블록은 변수 차원을 제대로 다룰 수 없습니다. 다변량 시계열 데이터에는 변수 간 상호작용 정보도 중요하기 때문에 모던 컨볼루션에 변화가 필요했습니다.

Time Series Related Modifications

CV에서는 3채널 RGB 이미지를 각 픽셀에서 D 차원 임베딩으로 투영하면서 각 채널의 정보를 믹싱합니다. 그러나 이런 단순한 임베딩 방식은 시계열 데이터의 변수 믹싱에 적절하지 않습니다. 왜냐하면: 이미지의 채널 간 차이보다 시계열의 각 변수 간의 차이가 크고; 변수간의 복잡한 의존성을 학습하기 어려우며; 변수 차원을 삭제하기 때문에 변수 간 의존성에 대한 후속 연구가 불가능합니다. 그래서 저자는 patchify variable-independent embedding을 제안합니다. 이는 각 변수를 합쳐 하나의 임베딩을 만드는 대신 변수별로 패치화(patchify) 한 후 임베딩을 만들어 변수 차원을 살리는 방식입니다.

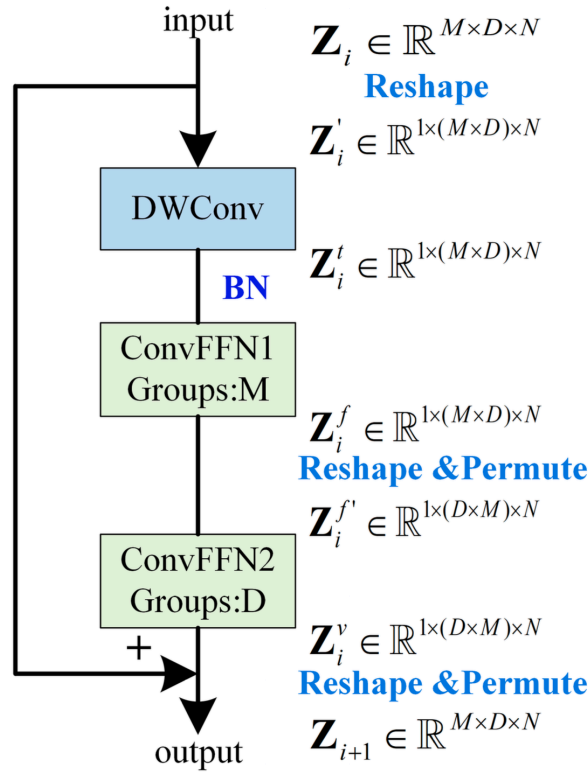
입력 시계열 데이터 $X \in \mathbb{R}^{M \times L}$ (M : 변수의 개수, L : 시계열의 길이)라고 할 때 각 변수 $x_i \in \mathbb{R}^{1 \times L}$ ($i=1, \dots, M$)를 적절히 패딩한 후에 패치 크기 PP , stride SS 로 NN 개의 패치로 변환(1D 컨볼루션)합니다. 이후 각 패치를 DD -차원의 임베딩 벡터가 되고 최종적으로 데이터는 $X_{emb} \in \mathbb{R}^{M \times D \times N}$ 이 됩니다. 여기서 M 은 변수 차원, DD 는 피쳐 차원, NN 은 시간 차원입니다.

차원에 관한 설명

변수 차원: 각 변수는 독립적으로 임베딩 되므로 이전과 똑같은 M 차원입니다.

시간 차원: 패치화를 하면서 $1 \times L$ 의 벡터가 $N \times P$ 인 행렬로 변했습니다. 이 때 이 행렬은 여러 연속되는 타임스텝이 묶인 P 차원 벡터가 N 개가 있는 구조이고 결국 이 N 개의 벡터들도 시간적으로 연속되기 때문에 벡터를 D 차원의 임베딩 벡터로 투영하면서 N 을 새로운 시간차원으로 본 것 같습니다.

피쳐 차원: NLP의 word embedding을 떠올려보면 단어 하나가 512차원, 768차원의 임베딩 벡터를 가집니다. 이 때 각 차원이 무엇을 의미하는지는 정확히 모르지만 단어의 의미정보를 가지고 있을 것이라고 생각하게 되는데요. 마찬가지로 시계열에서의 피쳐도 시계열이 가진 어떤 의미 정보를 가지고 있을 것으로 기대하여 D 차원의 임베딩으로 패치를 투영합니다. 즉, 위 임베딩 과정을 거치면 각 M 개의 변수가 N 만큼의 시간 단위를 가지는 D 개의 피쳐를 가지는 데이터로 변환됩니다.



(d) ModernTCN block

DWConv ModernTCN은 기존 모던 컨볼루션의 DWConv를 개조해 피쳐와 변수에 모두 독립적으로, 즉, 시점 정보만을 학습하도록 했습니다. 그리고 ERFs를 키우기 위해 커널 사이 즈릴 키워 시간 모델링 능력을 올렸습니다.

ConvFFN 기존 모던 컨볼루션에는 1층만 있던 ConvFFN를 두 층으로 디커플링했습니다. ConvFFN1은 각 변수의 피쳐 표현을 학습하기 위한 층이고, ConvFFN2는 각 피쳐의 변수 간 상호작용을 포착하기 위해 고안되었습니다.

따라서 DWConv, ConvFFN1, ConvFFN2는 각각 시간, 피쳐, 변수 차원의 상호작용을 따로따로 학습하게 됩니다. 여기에 스킵 커넥션이 하나 추가되어 한 블록을 통과하는 동안의 프로세스는 다음과 같이 표현됩니다:

$$\mathbf{Z}_{i+1} = \text{Block}(\mathbf{Z}_i) + \mathbf{Z}_i$$

여기서 i 는 블록 번호를 의미합니다. 전체적인 backbone의 구조는 이런 블록이 여러개 쌓인 구조이기 때문입니다.

아래 이미지는 각 컨볼루션 기반의 모델(ModernTCN, SCINet, MICN)의 ERFs를 확인하기 위해 마지막 은닉 상태 피쳐맵에서 가운데 있는 포인트에 영향을 미치는 부분을 시각화한 결과입니다. ks(kernel size)를 키운 ModernTCN의 구조가 가장 넓은 ERFs를 가진 것을 확인할 수 있습니다.

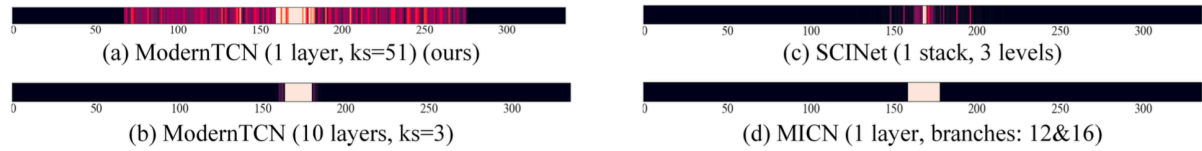


Figure 1: The Effective Receptive Field (ERF) of ModernTCN and previous convolution-based methods. A more widely distributed light area indicates a larger ERF. Our ModernTCN can obtain a much larger ERF than previous convolution-based methods. Meanwhile, enlarging the kernel size is a more effective way to obtain large ERF than stacking more small kernels.